

Derivative Products and Their Pricing — Interest Rate (IR) Module

Top 15 IR products — Advanced Quant Guide

Amit Kumar Jha, Quant Analyst at UBS

November 23, 2025

Contents

1	1. Overnight Indexed Swap (OIS)	2
2	2. Plain-Vanilla Interest Rate Swap (IRS)	10
3	3. Forward Rate Agreement (FRA)	17
4	4. Interest Rate Futures (Eurodollar / SOFR Futures)	24
5	5. Fixed-Coupon Bond (Government Bond)	31
6	6. Zero-Coupon Swap / Zero-Coupon Bond	39
7	7. European Swaption (Payer/Receiver)	48
8	8. Bermudan Swaption	58
9	9. Cap / Floor / Collar	71
10	10. Range Accrual Note	81
11	11. Inverse Floater	90
12	12. Callable & Puttable Bond	96
13	13. Constant Maturity Swap (CMS)	104
14	14. CMS Spread Option	111
15	15. Cross Currency Swap (CCS)	118



Production-Ready Python Code

```
1 from dataclasses import dataclass
2 from typing import List, Dict, Tuple
3 import numpy as np
4 from scipy.optimize import root_scalar
5 from enum import Enum
6
7 class DayCount(Enum):
8     ACT_360 = "ACT/360"
9     ACT_365 = "ACT/365"
10    THIRTY_360 = "30/360"
11
12 @dataclass
13 class IRSConfig:
14     """Immutable configuration for interest rate swap"""
15     notional: float
16     start_date: dt.datetime
17     maturity_date: dt.datetime
18     fixed_rate: float
19     pay_receive_fixed: str # 'pay' or 'receive'
20     fixed_freq: str = 'semiannual'
21     float_freq: str = 'quarterly'
22     fixed_daycount: DayCount = DayCount.THIRTY_360
23     float_daycount: DayCount = DayCount.ACT_360
24     float_spread: float = 0.0 # For basis swaps
25
26 class InterestRateSwap:
27     """Production-quality interest rate swap with full analytics"""
28
29     def __init__(self, config: IRSConfig):
30         self.config = config
31         self._validate()
32         self.schedule = self._build_schedule()
33         self.fixed_taus, self.float_taus = self._calculate_accruals()
34
35     def _validate(self):
36         if self.config.pay_receive_fixed not in ['pay', 'receive']:
37             raise ValueError("pay_receive_fixed must be 'pay' or 'receive'")
38         if self.config.notional <= 0:
39             raise ValueError("Notional must be positive")
40
41     def _build_schedule(self) -> List[Dict[str, dt.datetime]]:
42         """Build full swap schedule with reset and payment dates"""
43         schedule = []
44         freq_map = {'quarterly': 3, 'semiannual': 6, 'annual': 12}
45         fixed_months = freq_map.get(self.config.fixed_freq, 6)
46         float_months = freq_map.get(self.config.float_freq, 3)
47
48         # Fixed leg dates
49         fixed_dates = []
50         current = self.config.start_date
51         while current < self.config.maturity_date:
52             new_month = current.month + fixed_months
53             new_year = current.year + (new_month - 1) // 12
54             new_month = (new_month - 1) % 12 + 1
55             current = current.replace(year=new_year, month=new_month)
56             fixed_dates.append(current)
57
58         # Floating leg dates
```

3 3. Forward Rate Agreement (FRA)

Definition and Market Context

A Forward Rate Agreement (FRA) is a single-period forward contract that locks in an interest rate F for a future period $[T_1, T_2]$ on notional N . Unlike swaps, FRAs have only one payment at T_2 (or discounted settlement at T_1). FRAs are used to hedge future funding needs or express views on specific forward periods. They trade OTC with standard tenors like 3x6, 6x9, 12x15 (months).

Intuition: Time Travel Booking

An FRA is like booking a hotel room for next summer, today. You don't pay now, and you don't stay now. You just agree on the price.

- If hotel prices (rates) skyrocket by next summer, you win (you pay the agreed low rate).
- If prices crash, you lose (you are stuck paying the agreed high rate).

It isolates the risk of *future* rate movements for a specific window.

Quick Formula Sheet: FRA

- **Payoff:** $N \times \tau \times (F - K) \times \text{Discount}$
- **Forward Rate:** $F = \frac{1}{\tau} \left(\frac{P_1}{P_2} - 1 \right)$
- **Arb Check:** If $FRA > \text{Implied Fwd}$, Sell FRA / Buy Spot / Sell Forward.

Why Banks and Hedge Funds Use FRAs

- **Future Funding Hedge:** A corporation expecting to borrow in 3 months for 6 months can buy a 3x9 FRA to lock in today's forward rate.
- **Forward Curve Positioning:** Hedge funds trade FRAs against futures or swaps to exploit convexity differences and term premium.
- **Zero-Coupon Curve Building:** FRAs are primary instruments for bootstrapping the short end of the yield curve (6 months to 2 years).
- **Arbitrage:** FRA rates should equal implied forwards from the OIS curve. Deviations create arbitrage via cash-and-carry: borrow at OIS, invest in term deposit, lock in FRA.

Detailed Cashflow Mechanics

Settlement at T_2 (Standard):

$$\text{Payment} = N \cdot (F - K) \cdot \tau \quad \text{paid at } T_2$$

Settlement at T_1 (Discounted):

$$\text{Payment} = \frac{N \cdot (F - K) \cdot \tau}{1 + F \cdot \tau} \quad \text{paid at } T_1$$

Where $\tau = T_2 - T_1$ in day count fraction.

Pricing Mathematics (Detailed)

Forward Rate Implied from Curve:

$$F(0; T_1, T_2) = \frac{1}{\tau} \left(\frac{P(0, T_1)}{P(0, T_2)} - 1 \right) = \frac{P(0, T_1) - P(0, T_2)}{\tau \cdot P(0, T_2)}$$

Derivation: No-arbitrage condition: Invest $P(0, T_2)$ at T_2 = Invest $P(0, T_1)$ at T_1 , then reinvest forward to T_2 . Equating values yields the forward rate.

```

140     gamma = (self.annuity_at_expiry / (self.forward_rate * volatility * sqrt_t
141         )) * \
142         norm.pdf(d1)
143     vega = self.annuity_at_expiry * self.forward_rate * norm.pdf(d1) * sqrt_t
144     theta = -self.annuity_at_expiry * self.forward_rate * norm.pdf(d1) * \
145         (volatility / (2 * sqrt_t))
146
147 else: # receiver
148     d1 = (np.log(self.forward_rate / self.specs.strike_rate) +
149         0.5 * volatility**2 * self.time_to_expiry) / (volatility * sqrt_t)
150     d2 = d1 - volatility * sqrt_t
151
152     premium = self.annuity_at_expiry * (
153         self.specs.strike_rate * norm.cdf(-d2) -
154         self.forward_rate * norm.cdf(-d1)
155     )
156
157     delta = -self.annuity_at_expiry * norm.cdf(-d1)
158     gamma = (self.annuity_at_expiry / (self.forward_rate * volatility * sqrt_t
159         )) * \
160         norm.pdf(d1)
161     vega = self.annuity_at_expiry * self.forward_rate * norm.pdf(d1) * sqrt_t
162     theta = -self.annuity_at_expiry * self.forward_rate * norm.pdf(d1) * \
163         (volatility / (2 * sqrt_t))
164
165 return {
166     'Premium': premium * self.specs.notional,
167     'Delta': delta * self.specs.notional,
168     'Gamma': gamma * self.specs.notional,
169     'Vega': vega * self.specs.notional,
170     'Theta': theta * self.specs.notional,
171     'Forward_Rate': self.forward_rate,
172     'Annuity': self.annuity_at_expiry,
173     'D1': d1 if 'd1' in locals() else None,
174     'D2': d2 if 'd2' in locals() else None,
175     'model': 'black'
176 }
177
178 def price_normal(self, normal_vol: float) -> Dict[str, float]:
179     """
180     Price using Normal (Bachelier) model for negative rates.
181     Args:
182         normal_vol: Normal volatility in basis points (e.g., 75bp = 0.0075)
183     """
184     if self.time_to_expiry <= 0:
185         intrinsic = max(self.forward_rate - self.specs.strike_rate, 0) if \
186             self.specs.option_type == 'payer' else \
187             max(self.specs.strike_rate - self.forward_rate, 0)
188
189         return {
190             'Premium': intrinsic * self.annuity_at_expiry * self.specs.notional,
191             'Delta': self.annuity_at_expiry * self.specs.notional if intrinsic > 0
192                 else 0,
193             'Gamma': 0,
194             'Vega': 0,
195             'Theta': 0,
196             'model': 'expired'
197         }
198     }
199
200 d = (self.forward_rate - self.specs.strike_rate) / \

```